Your work, explained

FixFlow presentation and revision guide | 5 November 2026

Prepared from the repository at commit 9f4b082 and the recorded verification results. This guide explains the Member 3 web work completed in this chat, including the latest repair workflow integration.

Your main contribution in one sentence

I worked on the Vue web application: the screens, role-based navigation, frontend state, backend API integration, real-time refresh, validation, localization, and verification of the user experience.

A short opening you can practise

FixFlow helps people report and manage maintenance issues in shared spaces. My responsibility as Member 3 was the web frontend. I built and connected the experiences for reporters, technicians, administrators and shop managers. The interface changes according to the signed-in role. Reports come from the backend, and actions such as submitting a repair or confirming it use real API calls. I also worked on session handling, privacy, English and Chinese support, and testing. The latest integration supports the repair lifecycle from technician work through reporter confirmation and rating.

Remember these distinctions

- Frontend: collects input, presents data and calls APIs. Backend: enforces permissions, saves records and controls business transitions.
- A screen can exist before its backend data endpoint is available. Explain which parts are live and which still show an integration notice.
- A development payment simulation is not a real charge. Team approval of an agreement is not acceptance by every technician.

How to use this guide

Start with the stack, architecture and role chapters. Practise the repair and shop explanations next. Use the final questions and demo checklist for rehearsal. The source-file references let you show the relevant code if asked.

The technology stack and your folder

Why Vue and TypeScript appear together

Vue is the user-interface framework. TypeScript adds type checking to JavaScript. A .vue file is a Vue single-file component; it can contain a template, a TypeScript script and styles. A .ts file normally holds reusable logic such as an API service, type definition, store or helper. Mixing these files is intentional.

```
<script setup lang="ts">
// Reactive state and typed component logic
</script>
<template>
  <!-- The visible interface -->
</template>
<style scoped>
/* Styles for this component */
</style>
```

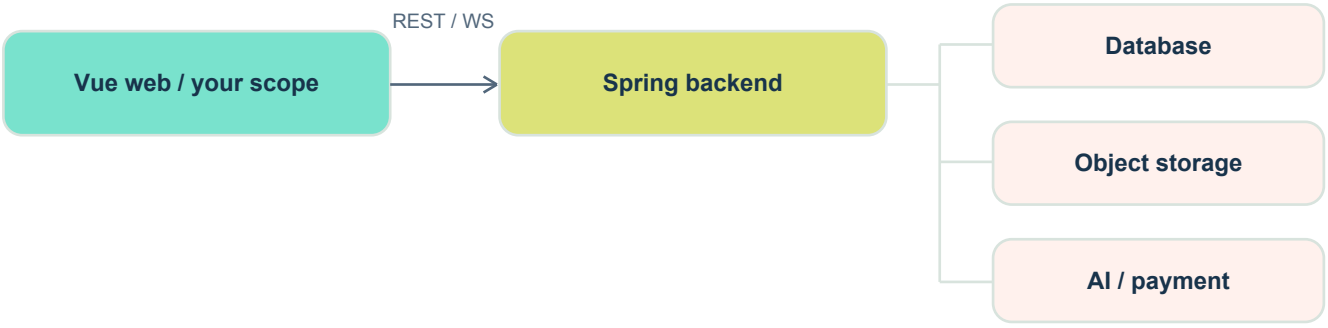
Tool	What it does in FixFlow
Vue 3	Renders components and updates the screen when reactive state changes.
TypeScript	Checks data shapes and catches type mistakes during development/build.
Vite	Runs the development server and creates the production bundle.
Pinia	Shares account, report, shop and real-time state across components.
Vue Router	Maps URLs to views and checks authentication/roles before navigation.
vue-i18n	Switches interface messages between English and Simplified Chinese.
Vitest + Vue Test Utils	Tests helpers, services, stores, route guards and component interactions.

Where your web work lives

Repository: /Users/apple/Desktop/fixflow. Web project: Frontend/Web. Views are pages; components are reusable pieces; services send requests; stores share state; types describe data; tests verify behaviour. Backend Java files belong to the server side, even when you read them to understand a contract.

Good presentation wording: "I integrated the backend contract in the web client." Do not describe a server algorithm or database implementation as frontend work.

How the whole application connects



Follow one request

- 1. A reporter fills in ReportFormView.vue and submits the form.
- 2. The view calls a service function in services/maintenance.ts.
- 3. services/api.ts sends the HTTP request, includes the session cookie, and obtains a CSRF token for a write.
- 4. The backend checks permissions and input, saves the record and returns JSON.
- 5. The frontend uses the returned report ID and data to open the saved report detail.
- 6. A later real-time event can trigger an authorized REST refresh.

What is an endpoint?

An endpoint is a server operation identified by an HTTP method and URL. For example, GET /api/reports/4 reads a report, while POST /api/reports/4/start asks the server to start work. The same URL with a different method can represent a different operation.

What is the API contract?

It is the agreement between client and server: path, method, request fields, response fields, permissions, limits and error codes. A DTO is the data shape used for that exchange. I inspected the Java controllers and DTOs, then matched the web payloads to them.

Why use service files?

They keep networking out of visual templates, centralize endpoint use and make requests easier to test. A component can ask startReport(id) without implementing headers, CSRF and JSON handling itself. The server remains the authority for whether the action succeeds.

Code to point to: src/services/api.ts, src/services/maintenance.ts and src/services/reportWorkflow.ts.

Read one action in the code

Start from the visible button

```
<button @click="perform('start')">Start work</button>
```

The template connects a click to the perform function in ReportWorkflowPanel.vue. The real button is shown only for the assigned technician in an allowed status. Busy/loading checks prevent accidental overlapping actions.

Follow the call into the service

```
export const startReport = (reportId: number) =>
  post<Report>(reportId, 'start');
```

export makes this function available to other files. reportId: number says the argument should be a number. post<Report> uses a TypeScript generic: the expected successful response has the Report shape. This is compile-time guidance, not proof that any arbitrary network response is valid.

The helper sends the HTTP request

The post helper builds /api/reports/{id}/start and calls request with method POST. The shared request function supplies credentials and the CSRF header, then awaits fetch. A Promise is a value that becomes available later; await pauses that asynchronous function until the operation finishes, without blocking the entire page.

Use the server result

```
saved = await api.startReport(reportId);
// After checking the request still belongs to this scope:
emit('saved', saved);
```

emit sends a component event to the parent. ReportDetailView listens with @saved and updates its report plus the maintenance store. Vue reacts to the changed state and renders the new status. On error, the panel shows the failure and does not pretend the report started.

The important type distinction

An interface such as Report describes expected fields in TypeScript. It does not create a database table or perform an API request. ref creates reactive component state; computed derives a value from other state; watch runs logic when selected values change. Keep that distinction clear when explaining .vue and .ts files.

A practical rehearsal

Open the panel file, find the Start work button, find perform, then open services/reportWorkflow.ts and follow startReport to post and request. Finish at the parent saved handler. This single path demonstrates that you understand the actual implementation.

Roles, pages and navigation

Users do not all have the same job. The web shows relevant links, and the router also protects direct URLs. The backend independently checks permissions on every request.

Role	Main experience	Important limits
Reporter	My reports, report creation/detail, sites, building suggestions, profile, shop	No management dashboard or product administration.
Technician	Technician workspace, contract requests, invitations, claim by ID and assigned-report work	No report creation or site-admin dashboard; site contracts are optional.
Staff	Operational dashboard, reports, assignments and site information	No unrestricted super-admin functions.
Site admin	Own-site reports, setup, team access, contracts and management actions	Site scope is enforced by the server.
Super admin	Administrative workspace with accessible-site selection and wider management	Sensitive/destructive actions still require authorization and explicit controls.
Shop manager	Product management at /shop/manage and profile	Customer catalogue/order routes redirect to management in the web.

Why hiding a button is insufficient

Someone can type a URL or call an API directly. Navigation filtering improves usability; route guards stop inappropriate page navigation; backend authorization protects the actual data and action. These are separate layers.

Where the rules are defined

- src/utils/navigation.ts: accountHome selects the useful landing page after sign-in.
- src/utils/workspaceAccess.ts: shared role-specific links and shop-manager route rules.
- src/router/index.ts: authenticated, guest and role guards.
- src/main.ts: clears shared state and reacts when account/role/site changes.

Example answer: "A reporter lands on My reports. A shop manager lands on the product list. Typing a management URL does not bypass the guard, and the API must still reject unauthorized requests."

Login, registration and session safety

Registration and approval

The registration form uses the backend signup policy for allowed self-registration roles and password rules. Reporters and technicians can self-register. A site-admin applicant receives reporter access while a super administrator reviews the application. Privileged roles are not granted just because someone chooses a label in the interface.

How login stays active

The backend creates the authenticated session and sends a cookie. Requests use credentials: include so the browser can send that cookie. On page reload, the frontend calls /api/auth/me to restore the signed-in account instead of assuming that a previous screen proves the user is still authenticated.

CSRF in plain language

Because browsers can automatically send cookies, a malicious website could try to make a user send an unwanted write request. The client gets a token from /api/auth/csrf and supplies it in X-CSRF-TOKEN for writes. The backend validates it. CSRF is different from XSS, which is unwanted script running inside a page.

What happens when a session expires?

- A protected API 401 means the current session is no longer authenticated. The web clears private state and returns the user to sign-in.
- A 403 means the account lacks permission. It should not automatically be treated as logout.
- Idle expiry concerns inactivity. Absolute expiry concerns the total authenticated lifetime. Both were checked using disposable local sessions.
- If logout fails on the server, the frontend reports the failure rather than pretending the server session has ended.

Avoiding an old-account response

Requests carry a session revision. An account/role/site transition advances it and aborts older requests. An old response must not overwrite the next account's data or let an old 401 sign out a newly logged-in account. The API layer also uses a 20-second request timeout.

Code: stores/auth.ts; services/auth.ts; services/api.ts; tests/requestSession.test.ts.

Reporting: from a form to a saved issue

What a reporter supplies

The form collects the site/building context, description, category, optional specialty, reporter urgency, optional urgency reason, optional room/address, visibility and optional photos. A building selection uses subsiteId; a room/address is separate. This avoids treating a free-text room as the actual building record.

What happens on submit

- Validate required values and DTO limits before sending.
- Use JSON for ordinary report intake, or multipart FormData when sending photos.
- Let the browser set the multipart boundary; do not manually invent its Content-Type.
- Show a busy state and any server error. Navigate using the report returned by the backend.
- If site setup is incomplete, guide an authorized administrator to configure labelled buildings.

Three urgency values

Value	Meaning
Reporter urgency	The person reporting the issue explains their assessment.
AI urgency	A separate result returned by the configured classification system.
Final urgency	The operational value, including an authorized override when used.

Keeping these separate preserves the original report and makes decisions understandable. The UI does not silently rewrite the reporter's words to look like an AI conclusion.

Privacy and joining an existing report

The backend can mask creator identity, address and reasons for viewers without the appropriate participation/access. The frontend displays null/masked fields as unavailable instead of reconstructing them from another cache. A reporter can join an existing issue through the join endpoint and refresh authorized participant information. This supports shared issues without requiring another duplicate report.

Limits include description 10,000 characters, optional address 512, category/specialty 128 and reporter reason 2,000. The server validates again. Code: ReportFormView.vue, ReportDetailView.vue, utils/reportIntake.ts and FORM_CONTRACTS.md.

The repair lifecycle you can demonstrate

```
open / routed / reopened / escalated
        | claim or authorized assignment
        v
     assigned --> in_progress
                      | submit repair description
                      v
          resolved_pending_confirmation
                      | reporter/operator confirms
                      v
                  confirmed --> rating
                      | issue returns
                      v
                   reopened
```

What was newly connected

The backend added ReportWorkflowController in commit 0fabdd5. The web then added a typed service, a repair workflow panel on report details and a technician claim-by-ID panel. Earlier statements that these lifecycle APIs were missing are now outdated.

Action	Who uses it / what it means
Claim	Technician supplies a report ID; backend checks contract, terms and claimability.
Start	Currently assigned technician starts assigned/reopened/escalated work.
Resolve	Assigned technician records repair details while work is in progress.
Confirm / reopen	Authorized reporter/operator verifies the repair or explains further work.
Escalate / override	Allowed roles supply an explanation; the backend records the transition or urgency change.
Rate / link	A confirmed resolution can be rated by eligible users; operators can link related reports.

Resolve is not the same as confirm

Resolving says "the technician has submitted the repair." Confirming says "the repair has been checked and accepted in the report workflow." A resolution has its own ID, description and time; a rating refers to that saved resolution ID.

Important remaining limit

Technicians can claim by ID, but the current backend does not expose a dedicated claimable-job list. Its normal report GET requires the technician to be assigned. The web therefore does not fabricate a browseable pool of unassigned jobs.

The lifecycle API map

Use this page when asked which endpoints you connected. All paths below start with /api/reports/{id}. The ID comes from the saved report, route or technician input.

Method + suffix	Request / purpose
POST /claim	No JSON payload; claim an eligible report as a technician.
POST /start	No JSON payload; start assigned work.
POST /resolve	{ description }; maximum 10,000 characters.
POST /escalate	{ reason }; maximum 2,000 characters.
PATCH /urgency	{ urgency, reason }; reason maximum 2,000.
GET /history	Status transitions, times and authorized/masked notes.
GET /resolutions	Recorded resolution IDs, technician IDs, descriptions and times.
POST /ratings	{ resolutionId, score, feedback }; score 1-5, feedback up to 2,000.
GET /ratings	Saved scores and feedback.
POST /links	{ linkedReportId, reason }; cannot link a report to itself.
GET /links	Related-report links filtered by the backend's access rules.

Related existing endpoints

POST /api/reports creates an issue. GET /api/reports reads a paginated site list. GET /api/reports/{id} reads detail. Existing POST suffixes include /assign, /classify, /join, /confirm and /reopen.

How failure is handled

The panel does not advance status before success. A server conflict leaves the current form and error visible. Reads for history, resolutions, ratings and links have separate failure reporting: a failed request is not shown as an empty successful list. Changing report/account/access scope invalidates late responses.

Code: services/reportWorkflow.ts, components/ReportWorkflowPanel.vue, components/technician/ClaimReportPanel.vue. Parent ReportDetailView coordinates busy states with assignment, review, classification and invitations.

Sites, buildings and account administration

Sites and subsites

A site is a managed location such as a school, hostel or hotel. A subsite is a labelled building/location within it. The web has accessible-site browsing, site detail/editing, reporting readiness, taxonomy and authorized technician-pool views. Super administrators can change site context; ordinary site users remain scoped to their account.

Building setup

Administrators can add/edit/archive labelled buildings. Existing reports retain location history when a building is archived. The editor moves keyboard focus into the form, returns it to the original Edit button on cancel, and focuses the saved confirmation after a successful save.

Missing-building suggestions: current reality

- A reporter can submit a missing-building suggestion from the report flow or /building-suggestions.
- The administrator can read pending suggestions, add the requested building and mark a request resolved.
- The reporter screen shows server submission receipts from the current visit. It clearly says this is not the complete saved history.
- The backend still rejects a second pending suggestion for the same reporter/site. It also lacks a reporter-accessible own-history endpoint. Those changes were requested from the backend team; the frontend cannot remove server restrictions.

Team access and rules

Authorized administrators can provision allowed accounts; super administrators review site-admin applications. The UI respects backend signup/capability rules. Type-wide site rules have management controls, and product administration has its own role restriction. Forms distinguish loading, success, validation failure and network failure.

A live example already checked

In the isolated verification database, the administrator created Science Annex from a reporter suggestion, edited it, resolved the request and reloaded. The building and resolution remained. A reporter attempt to resolve requests received 403. This is evidence for that scoped flow, not proof of every admin mutation.

Technicians, contracts and the agreement

Independent by default

A technician can register without a site and without site contracts. The single account.siteId value is not the list of contracted sites. Contract requests and actual site contracts are separate records and workflows. Trade/proficiency details are self-reported; saving them does not certify qualifications. Secure vetting-evidence upload and approval tracking remain unavailable.

Why the home-page details loop was changed

The home page previously mounted a fresh empty TradeProfile form on every visit. It looked as if the user had to register details again. The form was moved into an explicit, initially closed Profile section. Home now offers a Manage trade details link. Backend skill submission exists; retrieving all previously saved own skills still needs a read API.

Contract and work controls

- Technicians can submit site-contract requests and track the returned request list.
- Target-site administrators can review the requests; direct administrative contract creation is shown separately.
- Availability can be read/updated, invitations can be inspected, and candidates/ranking support authorized assignment.
- The web shows server ranking/fallback information; it does not invent an AI ranking.
- Removing a recorded relationship is an access/data operation, not a claim that a separate legal contract has been terminated.

Hiring marketplace: explain the limit honestly

/technician/marketplace exists and includes the functional contract-request/tracking panel. Live "sites open to hiring" listings need a published backend endpoint and an explicit hiring flag. A site's maintenance contract status does not prove that it is hiring.

Agreement approval versus individual acceptance

The team approved the English/Chinese participation document on 3 November. Its web URL is /agreements/technician/2026-11-03, and the old draft URL redirects there. The backend must configure the active version/document URL. Each technician's explicit acceptance is then recorded against that version; team approval does not automatically accept it for every account.

Code: TechnicianView.vue, ProfileView.vue, components/technician, TechnicianTermsPanel.vue and constants/technicianAgreement.ts.

Shop: customers, managers and payments

Customer flow

The shop loads published products from /api/shop/catalog. Search/category/stock filters use returned data. Adding an item updates the Pinia basket and shows a visible status message. The basket contains product IDs and quantities; the backend quote calculates the authoritative totals and current availability.

What happens at checkout

- 1. Validate shipping fields and request a quote.
- 2. Create the order using an idempotency key. An unchanged retry retains the same key to avoid accidental duplicate creation.
- 3. Use the returned payment provider/checkout URL. URLs are validated before navigation.
- 4. For the dev provider, show an explicit simulation modal: simulate successful payment or leave the order unpaid. The web calls the dev-complete API only after that choice.
- 5. Read order history/detail from the backend. A pending unpaid order has a separate cancellation confirmation.

What the simulation does and does not prove

It demonstrates the development payment state transition without charging money. It is not proof that a real payment provider is configured or that production settlement has been accepted. A provider webhook is server-to-server; it is not a browser button. Opening a return URL must not automatically trigger a development payment.

Shop-manager experience

Shop managers land on /shop/manage, with a semantic product table/list instead of cards. They can load/create/edit/deactivate products using the management API. Web route rules redirect this role away from customer catalogue/order pages. Backend rejection of direct manager purchase API calls remains a handoff item.

Money and product images

Prices use integer minor units with currency/precision returned by the service. For example only: 1,890 minor units with two decimal places means 18.90. The web must not assume every currency has two decimals. Product images use backend URLs, lazy loading and a visible fallback when unavailable; some verification catalogue images were still missing.

Code: stores/shop.ts; services/shop.ts and shopAdmin.ts; ShopView.vue, ShopOrderView.vue, ShopManagementView.vue and ProductImage.vue.

Real-time updates and stale-response protection

Why use WebSockets?

Maintenance changes can happen while another user is looking at the report. A cookie-authenticated WebSocket supplies small change events so the web can refresh without requiring a full manual reload.

The update sequence

```
Backend saves a report change
   -> event with eventId + report/site identifiers
   -> authorized subscribed web client receives it
   -> duplicate event IDs are ignored
   -> client rechecks access and fetches current REST data
   -> reactive state updates the screen
```

Events do not replace authorized report data

The client treats events as invalidation signals. It gets the report again through REST, where access is checked. This avoids treating an event payload as a permanent permission to see private report fields.

Reconnection and access changes

- Reconnect attempts restore only appropriate subscriptions and revalidate session/access.
- Logout stops the connection, clears subscriptions and clears shared state.
- Reassignment can revoke a technician's old report access. The web clears old private content and fetches only what is still authorized.
- A failure to reconnect is displayed as connection state; it is not a fabricated successful sync.

A race condition example

Imagine opening Report A, then switching to Report B before A's slow request finishes. Without a guard, A could overwrite B. Generation counters identify which scope started a request. If the scope changed, the response is discarded. Session revisions also protect transitions between accounts, not just between pages.

Your presentation answer

"The socket tells the UI something changed. REST supplies the current authorized record. I use session and request-generation checks so late data from an old account, site or report cannot overwrite the current screen."

Code: services/reportRealtime.ts, stores/realtime.ts, services/api.ts, stores/maintenance.ts and ReportDetailView.vue.

Photos and AI assistance

Report photos are separate from AI analysis

Ordinary photo upload attaches evidence to a report. It does not automatically send the image to an external AI provider. The web has authorized list/upload/delete controls and a signed-link refresh action. Upload validation allows up to five photos per report and 5 MiB each, with the supported image types checked again by the server.

Why temporary signed links?

The backend gives the browser a temporary download URL. Storage access credentials remain on the server. When a link expires, the web asks the authorized photo endpoint for a new one. The browser must not rewrite the signed host/path/query because that can invalidate the signature.

Storage setup	Meaning
Native/local MinIO	MinIO manages persistent objects on local disk; backend returns browser-reachable HTTP(S) signed URLs.
Docker MinIO	The same object API, with persistent Docker volumes and separate internal/public endpoints.
MINIO_ENABLED=false	Current backend chooses volatile memory storage, not a local-disk driver. memory:// URLs cannot render as browser images.

AI features connected in the web

- Manual classification shows AI output separately from reporter input and operational urgency.
- Image analysis requires explicit consent, checks the file and polls/refetches a background job status.
- The report assistant requires explicit consent before sending a question and authorized report context to the configured provider. Chat is cleared when scope/access changes.
- Provider errors are displayed. The UI does not manufacture an AI answer when a provider is unavailable.

What is still unverified?

Persistent photo upload/download/restart testing in both deployments and a permitted live external AI-provider run remain open. Existing interface/service tests prove request handling and failure states, not those external services' production readiness. Never put MinIO secrets or AI keys in VITE_* browser variables.

Code: ReportPhotosPanel.vue, ReportVisionPanel.vue, ReportClassificationPanel.vue, ReportAssistantPanel.vue; PHOTO_STORAGE.md.

Design, localization and accessibility

The visual direction

The requested palette is coral #e27979, mint #79e2cd and citron #dce279. The web uses these accents with warm neutral surfaces, clear cards/forms, a shared navigation shell and responsive layouts. Later text-contrast changes made body text and action labels easier to read without changing the requested brand accents.

English and Simplified Chinese

The approved languages are English and Chinese. Interface messages go through the locale layer, with a saved language choice. Dates and currency formatting use the locale. Backend identifiers, API enum values, user names, report descriptions and other user-authored text are not silently translated into different request values.

Accessibility work that was actually done

- Labels for form inputs, meaningful buttons and status/error feedback.
- A skip link, page-content focus after navigation and keyboard access to the mobile navigation.
- Escape/close behaviour and focus restoration for the navigation dialog.
- Contained horizontal scrolling for wide tables instead of forcing the whole page off-screen.
- Building-edit focus moves into the form, cancel returns to the initiating button and saved confirmation receives focus.
- Targeted narrow-layout and text-contrast checks, including selected English/Chinese views.

What you should not claim

These checks do not establish full WCAG conformance or complete screen-reader acceptance for every page. Wider assistive-technology, non-text contrast and target-size review remain open. Say "I improved and tested selected keyboard and responsive behaviours" rather than "every accessibility requirement is certified."

How reusable components help

Shared buttons, fields, labels, urgency chips, layouts and image fallbacks reduce repeated work and keep interactions consistent. A change to a shared component can improve several screens, but still needs checks where it affects role or data behaviour.

Code: src/i18n/index.ts, src/i18n/zh-CN.json, layouts/, components/ui/ and styles/. Evidence: ACCESSIBILITY_REVIEW.md.

Testing: what the numbers really mean

Latest recorded results are from 4 November 2026. They are evidence recorded for that code state, not a new test run performed just to produce this guide.

Evidence	Result	What it establishes
Web automated suite	389 passed; 7 live checks skipped by default	Client helpers, stores, transport, route rules and component interactions.
Backend workflow suite	8 passed	Server workflow authorization, transitions and related rules tested by the backend integration suite.
Opt-in live service suite	7 passed in the earlier separate local run	Production web service functions communicating with an isolated running backend.
Production build	Passed	Type checking and production bundling succeeded.
Manual live browser checks	Recorded scoped cases	Actual views and selected backend flows worked together.

Examples worth explaining

- A reporter management request returns 403 while the session remains valid.
- An old account response cannot clear or overwrite the next account's state.
- Private report fields disappear after reassignment/access revocation or session expiry.
- A failed update preserves form values; a failed list request does not show a false empty result.
- Checkout retries reuse the same key; an invalid payment URL preserves the basket.
- The repair panel uses server-returned resolution IDs and ignores old history after switching reports.

The live repair check

A new disposable report was claimed through an API fixture setup. In the browser, the technician started and resolved it. The reporter then confirmed it and submitted a rating. Reloading preserved status, resolution, rating and history. This did not mutate real maintenance records.

Important limits

The complete cross-role/privacy/browser matrix is still open. Operator escalation, urgency and link browser mutations have not all been manually accepted. There is no committed standalone automated browser regression runner. Source wiring, unit tests, integration tests and a live demo answer different questions.

Run it and rehearse a six-minute demo

Frontend startup

```
cd /Users/apple/Desktop/fixflow/Frontend/Web
npm ci  # only when dependencies need installing
VITE_API_URL= API_PROXY_TARGET=http://127.0.0.1:8080 \
  npm run dev -- --host 127.0.0.1 --port 8081
```

This example assumes the team backend runs on 8080. Use its actual address; the isolated verification backend used 8082. Keeping VITE_API_URL empty uses the same-origin Vite proxy for /api and /ws. A stale .env API URL can send requests to the wrong backend. Start the backend separately using the team's instructions.

Time	Show / explain
0:00-0:45	Your role, stack and frontend/backend boundary.
0:45-1:45	Reporter: create/view an issue, building selection and urgency.
1:45-3:15	Technician: assigned issue -> start -> resolution. Reporter: confirm and rate.
3:15-4:15	Admin: building suggestions/setup and role-restricted pages.
4:15-5:15	Shop: visible basket feedback, quote/order, clearly labelled dev simulation. Shop manager: product table.
5:15-6:00	English/Chinese, one security/test example, and honest remaining work.

Before presenting

- Use approved disposable demo accounts for each role; do not display passwords or private real records.
- Prepare an eligible contracted technician and a fresh report. Ensure active terms are configured consistently with the demo.
- Use the development payment provider if demonstrating simulation. Do not present a production charge as a test.
- Have screenshots ready in case the network, provider or server is unavailable.

If something fails

Connection refused: check that the frontend/backend process is running on the shown port. 401: sign in again. 403: check role/site/assignment. 409: the business state changed or duplicates are restricted. 404 on new workflow routes: confirm the running backend includes the new controller. CSRF failure: check the backend target/cookie context and reload the sign-in flow.

Code walkthrough: files worth knowing

You do not need to memorize every line. Know where responsibility lives and be able to follow one action from a button to its service, response and UI update.

File under Frontend/Web/src/	What to explain
router/index.ts	URL -> page; guest/authenticated/role checks.
utils/navigation.ts + workspaceAccess.ts	Role landing pages and relevant links.
services/api.ts	Credentials, CSRF, JSON/errors, timeout and request-session cancellation.
stores/auth.ts	Signed-in user, hydration, logout and scope changes.
stores/maintenance.ts	Accessible sites, selected site, reports and shared refresh.
views/app/ReportFormView.vue	Input, building selection, validation and saved-report navigation.
views/app/ReportDetailView.vue	Authorized detail and coordination of action panels.
services/reportWorkflow.ts	The eleven newly published lifecycle mappings.
components/ReportWorkflowPanel.vue	Status/role controls, reasons, ratings, links and history.
components/technician/ClaimReportPanel.vue	Claim by explicit ID and navigate after server success.
services/reportRealtime.ts + stores/realtime.ts	Events, subscriptions, deduplication, reconnect and REST refresh.
views/app/ShopView.vue + stores/shop.ts	Catalogue, basket feedback, quote/order and dev simulation.
views/app/ShopManagementView.vue	Management table with API-backed editing/deactivation.
i18n/index.ts + i18n/zh-CN.json	Saved locale and message translation.

Useful verification commands

```
VITE_API_URL= npx vitest run
VITE_API_URL= npm run build
# Live checks need explicit disposable fixture environment:
VITE_API_URL= npm run test:live
```

Documentation map: ENDPOINT_AUDIT.md inventories controller mappings; FORM_CONTRACTS.md records DTO boundaries; VERIFICATION.md records live/test evidence; MEMBER3_ACCEPTANCE.md tracks outstanding requirements; BACKEND_TEAM_MESSAGE.md contains the unsent integration request.

Questions you may be asked: foundations

1. What exactly was your responsibility?

The Vue web frontend: reusable screens/components, role navigation, shared state, API integration, error/loading feedback, privacy cleanup, localization and verification. The backend owns persistence, authentication enforcement, business transitions and provider integration.

2. Why do you use both .vue and .ts files?

.vue packages a component's template, logic and styles. TypeScript in the script improves type checking. Separate .ts files hold services, stores, interfaces and helpers reused by components.

3. Why Pinia instead of putting everything in components?

Account, selected-site/report and basket state is shared across pages. Pinia gives it a clear owner. Temporary form fields can stay local so unrelated views do not accidentally share them.

4. What happens after a user clicks Submit?

The component checks input, calls the typed service, the shared API layer sends credentials/CSRF, the backend validates and saves, and the UI uses the returned record. Failure produces an error instead of a pretend success.

5. How do you stop a reporter opening an admin dashboard?

Filter their navigation, guard the route even for a typed URL, and rely on backend authorization for the actual operation. Hiding a link alone cannot secure an API.

6. Why can a technician have no site?

Technicians are independent by default. Site contracts are separate relationships and are optional. A null account.siteId does not mean that all contract records are empty.

7. Where do products, prices and reports come from?

The backend APIs. The web should not invent business records, stock or payment success. Fixed interface text, role/status enums and authored agreement content are different from pretending to have live database data.

8. How do you know your frontend matches the backend?

Inspect controller paths/methods, DTO fields/limits, response types and role checks. Match them in service functions, test payloads and failures, then verify selected flows against the running backend.

Questions you may be asked: security and workflow

9. What is the difference between 401, 403 and 409?

401 means authentication is missing/expired. 403 means the account lacks permission. 409 means the requested operation conflicts with the current business state, such as an already claimed report or an existing rating.

10. Why include a CSRF token if there is a session cookie?

The cookie identifies the session, but browsers can send it automatically. A validated CSRF token helps prevent another website from causing an unwanted authenticated write.

11. What prevents private data from remaining after logout or reassignment?

Clear stores and subscriptions, invalidate pending request generations, recheck access and fetch authorized REST data. Late responses from the old scope are discarded.

12. Why not update the screen directly from a WebSocket event?

An event says something changed. The client refetches authorized REST data because permissions may have changed too. Event IDs prevent duplicate notifications from causing repeated processing.

13. What is the difference between resolving and confirming?

The technician submits a resolution; the issue waits for confirmation. An eligible reporter/operator then confirms it was fixed, or reopens it with a reason. Ratings use a recorded resolution after confirmation.

14. Why is claim-by-ID available without a job marketplace list?

The claim action is published, but a dedicated claimable queue is not. The current detail endpoint only lets a technician read a report once assigned. I connected the action without making up a list the server does not supply.

15. Does accepting the agreement for the team accept it for every technician?

No. Team approval publishes the content. The backend must select the active version, and each technician explicitly accepts that version. The web uses the backend status/history for enforcement and records.

16. Why can't reporters submit multiple pending building suggestions yet?

The backend explicitly rejects a second pending request for the same user/site. The new screen is available, but that rule and a reporter own-history endpoint need backend changes. A frontend-only workaround would misrepresent what the server saves.

Questions you may be asked: shop and delivery

17. What stops checkout creating duplicate orders?

The request uses an idempotency key. If the unchanged checkout must be retried after a network failure, the client retains that key so the backend can identify the same attempt.

18. Does a simulated payment charge real money?

No. The dev provider demonstrates a payment state transition. The user sees an explicit simulation step. Production payment processing and webhooks are separate backend/provider concerns.

19. Why use integer minor units for money?

It avoids treating decimal prices as imprecise floating-point amounts in client arithmetic. Formatting uses currency and minor-unit precision returned by the backend, rather than assuming all currencies use cents.

20. Why did shop owners see the customer shop before?

Their old landing route pointed to /shop. It now points to /shop/manage, with a product table and customer-route redirection. Server-side rejection of direct purchase requests by that role is also requested from the backend team.

21. Are photo storage and AI fully verified?

The web controls, payloads and error/consent handling are implemented. Persistent MinIO deployment acceptance and permitted live external AI verification are still open. Ordinary photos are not automatically sent to AI.

22. What do the 389 passing tests prove?

The recorded web suite covers many helpers, stores, route rules and component interactions. The eight backend workflow tests and selected live checks provide different evidence. None alone proves every browser, role and production integration is ready.

23. Is the application fully accessible?

Selected keyboard, focus, contrast and responsive behaviours were improved and checked. Wider screen-reader and full conformance review remain outstanding.

24. What would you improve next?

Finish the hiring/queue and reporter suggestion-history contracts, load saved technician skills, complete storage/provider testing and the broader role/accessibility matrix, and review the remaining operator workflows. Keep the audit current as backend contracts change.

Your final rehearsal sheet

Five things to remember

- My scope: the web experience and integration. The backend enforces permissions and business rules.
- Data flow: component -> service -> shared API layer -> backend -> returned record -> reactive UI.
- Security: role links + route guards + backend authorization + session/CSRF/scope cleanup.
- Repair: claim/assign -> start -> resolve -> confirm -> rate; reopen and escalation are separate transitions.
- Evidence: recorded 389 web tests, 8 backend workflow tests, production build and scoped live checks; remaining gaps are explicit.

A two-minute presentation script

My part of FixFlow is the Vue web application. I organized it into views, reusable components, API services and Pinia stores. Vue handles the reactive interface, TypeScript checks data shapes, and Vue Router controls page access.

The application changes by role. Reporters create and follow maintenance issues. Technicians use an independent workspace and work on reports assigned or claimed through the service. Administrators manage sites, buildings, access and contracts. Shop managers go directly to the product-management list.

For reporting, I connected building selection, report creation, privacy-aware detail, participation, classification and the repair workflow. A technician can start work and submit a resolution. The reporter can confirm it and rate the saved repair. These actions use backend responses; the UI does not invent the final status.

I also handled session restoration, CSRF-protected writes, role navigation and cleanup when account or report access changes. Real-time events trigger authorized refreshes. English and Chinese interface support, visible basket feedback and an explicit development payment simulation improve the experience.

The recorded web suite has 389 passing tests, and the new backend workflow suite has eight. I also verified a disposable repair through technician resolution, reporter confirmation and rating in the live browser. Remaining work includes live hiring listings, full building-request history and multiple pending suggestions, saved technician-skill retrieval, storage/provider verification and wider acceptance checks.

If you do not know an answer

Explain the part you can show in code, identify whether the question concerns frontend or backend responsibility, and check the relevant contract. A precise limitation is a stronger answer than claiming a feature works without evidence.